

Dynamic Programming: MCM and Partition DP

A Problem-Solving Companion to Striver's A2Z DSA Sheet

Maruri Saikiran

June 29, 2026

Contents

1	Dynamic Programming: MCM and Partition DP	1
1.1	Interval DP: Searching Over the Split Point	1
1.2	Matrix Chain Multiplication	3
1.3	Minimum Cost to Cut a Stick	5
1.4	Burst Balloons	7
1.5	Evaluate Boolean Expression to True	9
1.6	Palindrome Partitioning II	11
1.7	Partition Array for Maximum Sum	12

Chapter 1

Dynamic Programming: MCM and Partition DP

Every problem in this chapter asks the same question about a range: if I must split this range somewhere, which split point minimizes (or maximizes) the cost of the two halves plus the cost of joining them? The split point itself becomes the thing being searched over, inside an outer loop over every possible range.

1.1 Interval DP: Searching Over the Split Point

Every problem in this chapter shares a state shape new to this book: $f(i, j)$, the optimal answer for the **range** (interval) from index i to index j – and the recurrence searches over every possible split point k *inside* that range, combining the optimal answers for (i, k) and (k, j) plus a cost specific to joining at k :

$$f(i, j) = \min_{i \leq k < j} \left(f(i, k) + f(k+1, j) + \text{joinCost}(i, k, j) \right).$$

Why This Needs a Third Loop, Not Just a Third Dimension

Every DP table so far in this book has been filled by iterating over *indices*. Interval DP tables are filled by iterating over **range lengths** (smallest ranges first, since $f(i, j)$ depends on strictly smaller sub-ranges), and for each range, an **additional inner loop over every split point** k inside it. This third loop is what pushes naive interval DP to $O(n^3)$ even after memoization removes the exponential blowup – there are $O(n^2)$ ranges, each requiring an $O(n)$ scan over split points.

How the Six Problems in This Chapter Vary the Template

- **Matrix Chain Multiplication (Section 1.2)**: the foundational version – joinCost is the scalar multiplication cost of combining two matrix products.
- **Minimum Cost to Cut a Stick (Section 1.3)**: identical shape, with joinCost being the current stick segment's length.
- **Burst Balloons (Section 1.4)**: the same shape, but with a crucial reframing – k represents the *last* balloon burst in the range, not an arbitrary split, which is what makes the cost formula well-defined.
- **Evaluate Boolean Expression (Section 1.5)**: k ranges over operator *positions*, and each range tracks a *pair* of values (ways to make it **True**, ways to make it **False**) instead of a single number.
- **Palindrome Partitioning II (Section 1.6)**: uses an interval-DP table only as a **precomputation** (which substrings are palindromes), feeding into a separate, simpler 1D ending-here cut-counting DP.

- **Partition Array for Maximum Sum (Section 1.7):** a simplified partition DP where the split points are bounded (every partition has length at most k), removing the need for a full $O(n)$ search over all split points.

1.2 Matrix Chain Multiplication

Problem Statement. Given an array arr of $n + 1$ integers representing n matrices, where the i -th matrix has dimensions $arr[i - 1] \times arr[i]$, find the **minimum number of scalar multiplications** needed to compute the product of all n matrices (matrix multiplication is associative, so the order of multiplying *pairs* can be chosen freely).

Pattern Recognition

“Minimum cost to fully combine a chain, where combining two adjacent groups costs something dependent on their boundaries” is this chapter’s foundational interval DP: for matrices i through j , try every possible point k to split the chain into a left group and a right group, multiply those two groups’ results together (this costs $arr[i - 1] \times arr[k] \times arr[j]$ scalar multiplications, since the left group reduces to one $arr[i - 1] \times arr[k]$ matrix and the right to one $arr[k] \times arr[j]$ matrix), and take the best split.

Deriving the recurrence

Let $f(i, j)$ be the minimum scalar multiplications to multiply matrices i through j (using 1-indexed matrix numbers, where matrix i has dimensions $arr[i - 1] \times arr[i]$). For a single matrix, $f(i, i) = 0$ (nothing to multiply). For $i < j$, try every split point k from i to $j - 1$ (matrices $i..k$ form the left group, $k+1..j$ the right group):

$$f(i, j) = \min_{i \leq k < j} \left(f(i, k) + f(k+1, j) + arr[i-1] \cdot arr[k] \cdot arr[j] \right).$$

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int mcmRecursive(int i, int j, vector<int>& arr) {
5     if (i == j) return 0;
6
7     int minCost = INT_MAX;
8     for (int k = i; k < j; k++) {
9         int cost = mcmRecursive(i, k, arr) + mcmRecursive(k + 1, j, arr
10             ) +
11             arr[i - 1] * arr[k] * arr[j];
12         minCost = min(minCost, cost);
13     }
14     return minCost;
15 }
16
17 int main() {
18     vector<int> arr = {40, 20, 30, 10, 30};
19     int n = arr.size() - 1;
20     cout << mcmRecursive(1, n, arr) << endl; // 26000
21     return 0;
22 }
```

Approach 1 Complexity

Time. Exponential – roughly $O(2^n \cdot n)$ from the Catalan-number-sized search space of parenthesizations. **Space.** $O(n)$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int mcmMemo(int i, int j, vector<int>& arr, vector<vector<int>>& memo)
5 {
6     if (i == j) return 0;
7     if (memo[i][j] != -1) return memo[i][j];
8
9     int minCost = INT_MAX;
10    for (int k = i; k < j; k++) {
11        int cost = mcmMemo(i, k, arr, memo) + mcmMemo(k + 1, j, arr,
12            memo) +
13            arr[i - 1] * arr[k] * arr[j];
14        minCost = min(minCost, cost);
15    }
16    return memo[i][j] = minCost;
17 }
18
19 int main() {
20     vector<int> arr = {40, 20, 30, 10, 30};
21     int n = arr.size() - 1;
22     vector<vector<int>> memo(n + 1, vector<int>(n + 1, -1));
23     cout << mcmMemo(1, n, arr, memo) << endl; // 26000
24     return 0;
25 }

```

Approach 2 Complexity

Time. $O(n^2)$ distinct (i, j) states, each doing an $O(n)$ scan over k : $O(n^3)$. **Space.** $O(n^2)$ memo + $O(n)$ recursion stack.

Approach 3: Tabulation

Filling Order: by Increasing Range Length, Not by Index

$f(i, j)$ depends only on *strictly smaller* ranges (i, k) and $(k+1, j)$, both contained within $[i, j]$. So the table must be filled in order of increasing range length $(j - i)$, not in simple increasing i or j order – the first new fill-order strategy in this book.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int mcmTabulation(vector<int>& arr) {
5     int n = arr.size() - 1;
6     vector<vector<int>> dp(n + 1, vector<int>(n + 1, 0));
7
8     for (int len = 1; len < n; len++) {
9         for (int i = 1; i + len <= n; i++) {
10             int j = i + len;
11             dp[i][j] = INT_MAX;
12             for (int k = i; k < j; k++) {
13                 int cost = dp[i][k] + dp[k + 1][j] +
14                     arr[i - 1] * arr[k] * arr[j];
15                 dp[i][j] = min(dp[i][j], cost);
16             }
17         }
18     }
19 }

```

```

16         }
17     }
18 }
19 return dp[1][n];
20 }
21
22 int main() {
23     vector<int> arr = {40, 20, 30, 10, 30};
24     cout << mcmTabulation(arr) << endl; // 26000
25     return 0;
26 }

```

Dry run

Dry Run. Take $arr = [40, 20, 30, 10, 30]$ (4 matrices: 40×20 , 20×30 , 30×10 , 10×30).

For the full range $(1, 4)$, the best split is at $k = 3$: multiply matrices 1..3 combined ($f(1, 3) = 14000$, itself optimally split at $k = 1$ inside that sub-range) with matrix 4 alone ($f(4, 4) = 0$), joined at cost $arr[0] \cdot arr[3] \cdot arr[4] = 40 \cdot 10 \cdot 30 = 12000$. Total: $f(1, 4) = 14000 + 0 + 12000 = 26000$, matching the expected answer (the alternative splits at $k = 1$ and $k = 2$ both give strictly larger totals, 36000 and 69000 respectively).

Approach 3 Complexity

Time. $O(n^3) - O(n^2)$ ranges, each scanning $O(n)$ split points. **Space.** $O(n^2)$ for the table. There is no meaningful space-optimized version: every cell (i, j) genuinely depends on arbitrarily distant smaller ranges, not just a fixed nearby row, so the full 2D table must be retained.

Key Takeaway

Interval DP's defining feature – filling by range length, with an inner search over every split point – appears here for the first time in this book, and every other section in this chapter reuses this exact fill order with only the cost formula changed.

1.3 Minimum Cost to Cut a Stick

Problem Statement. Given a stick of length n and an array *cuts* of positions where cuts must be made (in any order), where the cost of making one cut equals the **current length** of the piece being cut, find the minimum total cost to perform all cuts.

Pattern Recognition

This problem is Section 1.2's interval template with the join cost changed from a triple product to a simple length difference: choosing which cut to make **first** among a range of pending cuts is exactly the same kind of “pick a split point k ” decision.

Deriving the recurrence

Add boundary markers 0 and n to *cuts*, then sort: $points = [0, c_1, c_2, \dots, c_m, n]$. Let $f(i, j)$ be the minimum cost to make every cut strictly between $points[i]$ and $points[j]$ (i.e., the original cuts at sorted indices $i+1, \dots, j-1$), on a stick segment currently spanning $[points[i], points[j]]$. Base case: $f(i, i+1) = 0$ (no cuts pending between adjacent points – nothing to do). For

$j > i + 1$, try every pending cut k as the **first** one made on this segment:

$$f(i, j) = \min_{i < k < j} \left(f(i, k) + f(k, j) + (points[j] - points[i]) \right).$$

The join cost $points[j] - points[i]$ is constant across every choice of k (cutting first, while the segment is still its full original length) – only the *choice of which k to cut first* affects how the two resulting sub-problems are structured.

C++ implementation (memoization)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int cutStickMemo(int i, int j, vector<int>& points, vector<vector<int
  >>& memo) {
5     if (j - i <= 1) return 0; // no cuts pending between adjacent
6     // points
7     if (memo[i][j] != -1) return memo[i][j];
8
9     int best = INT_MAX;
10    for (int k = i + 1; k < j; k++) {
11        int cost = cutStickMemo(i, k, points, memo) +
12                  cutStickMemo(k, j, points, memo) +
13                  (points[j] - points[i]);
14        best = min(best, cost);
15    }
16    return memo[i][j] = best;
17 }
18
19 int minCostToCutStick(int n, vector<int>& cuts) {
20     vector<int> points = cuts;
21     points.push_back(0);
22     points.push_back(n);
23     sort(points.begin(), points.end());
24
25     int m = points.size();
26     vector<vector<int>> memo(m, vector<int>(m, -1));
27     return cutStickMemo(0, m - 1, points, memo);
28 }
29
30 int main() {
31     vector<int> cuts = {3, 5, 1, 4};
32     cout << minCostToCutStick(7, cuts) << endl; // 16
33 }

```

Dry run

Dry Run. Take $n = 7$, $cuts = [3, 5, 1, 4]$. Boundary-augmented and sorted: $points = [0, 1, 3, 4, 5, 7]$.

The optimal ordering cuts at position 3 first, costing the full stick length 7, splitting into segment $[0, 3]$ (one pending cut at 1) and segment $[3, 7]$ (two pending cuts, at 4 and 5). Segment $[0, 3]$ costs 3 for its single cut. Segment $[3, 7]$ optimally cuts at 5 first (cost $7 - 3 = 4$), splitting into $[3, 5]$ (pending cut at 4, cost 2) and $[5, 7]$ (no pending cuts). Total cost: 7 (cut at 3) + 3 (cut at 1) + 4 (cut at 5) + 2 (cut at 4) = 16 , matching $f(0, 5) = 16$.

Complexity Analysis

Time Complexity. $O(m^3)$ where m is the number of cuts (plus boundaries) – $O(m^2)$ ranges, each scanning $O(m)$ split points. **Space Complexity.** $O(m^2)$ for the memo table.

Key Takeaway

The choice of *which* cut to make first only matters for how the remaining sub-problems are structured – the cost of making any cut on a segment of fixed current length is fixed regardless of which pending cut is chosen, which is exactly what makes the join-cost term constant across all choices of k in this recurrence.

1.4 Burst Balloons

Problem Statement. Given n balloons with values $nums$, bursting balloon i (whose current left neighbor is $left$ and right neighbor is $right$, among balloons not yet burst) earns $left \times nums[i] \times right$ coins, and the two neighbors become adjacent. Burst **all** balloons, in any order, to **maximize** total coins earned.

Pattern Recognition

“Choose an order to remove all items, where each removal’s value depends on its current neighbors” looks at first like it needs to track which specific balloons remain at every step – an exponential state. The fix, and the entire point of this section, is choosing k to represent which balloon is burst **last** within a range, not first, which keeps the boundary balloons’ identities fixed throughout.

Why “burst first” fails and “burst last” works

If k were chosen as the *first* balloon burst in range (i, j) , its reward would be $val[i] \times val[k] \times val[j]$ correctly – but bursting it merges i and j into direct neighbors *before* the rest of (i, j) is processed, which breaks the clean recursive split into two independent sub-ranges (the left and right remaining balloons are no longer separated by anything, so their sub-problems interact).

If instead k is chosen as the **last** balloon burst in range (i, j) (exclusive bounds; i and j themselves are never burst inside this sub-problem – they are the fixed boundary), then at the moment k is finally burst, *every other balloon strictly between i and j has already been removed*. So k ’s neighbors at that moment are exactly i and j themselves, regardless of the order in which the other balloons in (i, j) were burst. This makes k ’s reward **always** $val[i] \times val[k] \times val[j]$, independent of how the two resulting sub-ranges (i, k) and (k, j) are themselves resolved – which is exactly the independence interval DP needs.

Deriving the recurrence

Pad $nums$ with 1 at both ends: $val[0] = 1$, $val[1..n] = nums$, $val[n+1] = 1$ (bursting a boundary balloon should not be reduced by a missing neighbor, so a neighbor value of 1 is neutral in the product). Let $f(i, j)$ be the maximum coins from bursting every balloon strictly between i and j :

$$f(i, j) = \max_{i < k < j} \left(f(i, k) + f(k, j) + val[i] \cdot val[k] \cdot val[j] \right), \quad f(i, j) = 0 \text{ if } j \leq i + 1.$$

The answer is $f(0, n+1)$.

C++ implementation (memoization)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int burstMemo(int i, int j, vector<int>& val, vector<vector<int>>& memo
5 ) {
6     if (j <= i + 1) return 0;
7     if (memo[i][j] != -1) return memo[i][j];
8
9     int best = 0;
10    for (int k = i + 1; k < j; k++) {
11        int coins = burstMemo(i, k, val, memo) + burstMemo(k, j, val,
12            memo) +
13            val[i] * val[k] * val[j];
14        best = max(best, coins);
15    }
16    return memo[i][j] = best;
17 }
18
19 int maxCoins(vector<int>& nums) {
20     int n = nums.size();
21     vector<int> val(n + 2);
22     val[0] = 1;
23     val[n + 1] = 1;
24     for (int i = 0; i < n; i++) val[i + 1] = nums[i];
25
26     vector<vector<int>> memo(n + 2, vector<int>(n + 2, -1));
27     return burstMemo(0, n + 1, val, memo);
28 }
29
30 int main() {
31     vector<int> nums = {3, 1, 5, 8};
32     cout << maxCoins(nums) << endl; // 167
33     return 0;
34 }

```

Dry run

Dry Run. Take $nums = [3, 1, 5, 8]$, padded $val = [1, 3, 1, 5, 8, 1]$.

One optimal order: burst index 1 (value 1) first, neighbors 3, 5: $3 \times 1 \times 5 = 15$. Then burst index 2 (value 5), now neighbors 3, 8: $3 \times 5 \times 8 = 120$. Then burst index 0 (value 3), neighbors 1, 8: $1 \times 3 \times 8 = 24$. Then burst index 3 (value 8), neighbors 1, 1: $1 \times 8 \times 1 = 8$. Total: $15 + 120 + 24 + 8 = 167$. Read in reverse, this is exactly the recursive structure $f(0, 5)$ builds: value 8 (original index 3) is the **last** balloon burst overall, contributing $val[0] \times val[4] \times val[5] = 1 \times 8 \times 1 = 8$; within the remaining range, value 3 (index 0) is last, contributing $1 \times 3 \times 8 = 24$; within *that* remaining range, value 5 (index 2) is last, contributing $3 \times 5 \times 8 = 120$; and finally value 1 (index 1), alone, contributes $3 \times 1 \times 5 = 15$. Summing in this recursive order, $15 + 120 + 24 + 8 = 167$, matches $f(0, 5) = 167$.

Complexity Analysis

Time Complexity. $O(n^3)$ ($O(n^2)$ ranges, $O(n)$ split-point scan each). **Space Complexity.** $O(n^2)$ for the memo table.

Key Takeaway

When a removal (or insertion) order matters and each step's value depends on neighbors, ask whether reframing the choice as “which item is processed *last* in this range” (rather than first) fixes the range's boundary identities throughout – the single insight that turns an apparently sequential, order-dependent problem into a clean, order-independent interval DP.

1.5 Evaluate Boolean Expression to True

Problem Statement. Given a boolean expression string containing operands T and F separated by operators & (and), | (or), and ^ (xor), count the number of ways to fully parenthesize the expression so that it evaluates to **True** (answer modulo $10^9 + 7$).

Pattern Recognition

“Count the parenthesizations achieving a target outcome” is this chapter's interval template with two simultaneous twists: the split points k are restricted to **operator positions** only (operands themselves cannot be split), and each range tracks a **pair** of counts (ways **True**, ways **False**) rather than one number – because combining two ranges' outcomes through an operator needs to know *both* sub-outcomes' counts to compute the combined result correctly.

Deriving the combination rules

Let $f(i, j) = (\text{trueWays}, \text{falseWays})$ for the sub-expression spanning operand positions i to j . Base case: $i = j$ (a single operand): $f(i, i) = (1, 0)$ if $\text{expr}[i] = \text{T}$, else $(0, 1)$. For $i < j$, try every operator position k strictly between i and j , splitting into left $(i, k-1)$ and right $(k+1, j)$, and combine according to which operator sits at k :

- & (and): result is **True** only if *both* sides are **True**. $\text{trueWays} += L_T \cdot R_T$. $\text{falseWays} += L_T R_F + L_F R_T + L_F R_F$.
- | (or): result is **False** only if *both* sides are **False**. $\text{trueWays} += L_T R_T + L_T R_F + L_F R_T$. $\text{falseWays} += L_F R_F$.
- ^ (xor): result is **True** only if the sides *differ*. $\text{trueWays} += L_T R_F + L_F R_T$. $\text{falseWays} += L_T R_T + L_F R_F$.

Sum these contributions over every valid operator position k .

C++ implementation (memoization)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int MOD = 1e9 + 7;
4
5 pair<long long, long long> evalMemo(int i, int j, string& expr,
6                                     vector<vector<pair<long long, long
7                                         long>>>& memo) {
8     if (i == j) {
9         return expr[i] == 'T' ? make_pair(1LL, 0LL) : make_pair(0LL, 1
10            LL);
11     }
12     if (memo[i][j].first != -1) return memo[i][j];

```

```

11
12     long long trueWays = 0, falseWays = 0;
13     for (int k = i + 1; k < j; k += 2) { // operators sit at odd
14         // offsets
15         auto [lt, lf] = evalMemo(i, k - 1, expr, memo);
16         auto [rt, rf] = evalMemo(k + 1, j, expr, memo);
17
18         if (expr[k] == '&') {
19             trueWays = (trueWays + lt * rt) % MOD;
20             falseWays = (falseWays + lt * rf + lf * rt + lf * rf) % MOD;
21         }
22         else if (expr[k] == '|') {
23             trueWays = (trueWays + lt * rt + lt * rf + lf * rt) % MOD;
24             falseWays = (falseWays + lf * rf) % MOD;
25         }
26         else { // '^'
27             trueWays = (trueWays + lt * rf + lf * rt) % MOD;
28             falseWays = (falseWays + lt * rt + lf * rf) % MOD;
29         }
30     }
31     return memo[i][j] = {trueWays, falseWays};
32 }
33
34 int evaluateBooleanExpr(string expr) {
35     int n = expr.size();
36     vector<vector<pair<long long, long long>>> memo(n, vector<pair<long
37         long, long long>>(n, {-1, -1}));
38     return evalMemo(0, n - 1, expr, memo).first;
39 }
40
41 int main() {
42     cout << evaluateBooleanExpr("T|F&T^F") << endl; // example
43     // expression
44     return 0;
45 }

```

Dry run

Dry Run. Take $expr = "T^F|F"$ (positions: 0=T, 1=^, 2=F, 3=|, 4=F).

Base cases: $f(0,0) = (1,0)$, $f(2,2) = (0,1)$, $f(4,4) = (0,1)$. $f(0,2)$ (sub-expression "T^F", operator ^ at position 1): $L = (1,0)$, $R = (0,1)$. $trueWays = L_T R_F + L_F R_T = 1 \cdot 1 + 0 \cdot 0 = 1$. $falseWays = L_T R_T + L_F R_F = 1 \cdot 0 + 0 \cdot 1 = 0$. So $f(0,2) = (1,0)$. $f(0,4)$ (the full expression, one operator position to try, $k = 3$, operator |): $L = f(0,2) = (1,0)$, $R = f(4,4) = (0,1)$. $trueWays = L_T R_T + L_T R_F + L_F R_T = 1 \cdot 0 + 1 \cdot 1 + 0 \cdot 0 = 1$. $falseWays = L_F R_F = 0 \cdot 1 = 0$. Final answer: $trueWays = 1$ (the single way: $(T^F) | F$ evaluates to $True | False = True$).

Complexity Analysis

Time Complexity. $O(n^3)$ (the same interval-DP bound, with the operator-only stride halving the constant but not the asymptotic order). **Space Complexity.** $O(n^2)$ for the memo table (each cell storing a pair).

Key Takeaway

Whenever an interval DP's combination step has more than one possible *outcome* (here, **True** or **False**), and a later combination needs to know the count for *each* outcome to compute its own result, track a small tuple of values per cell instead of a single number –

the same upgrade Number of LIS made to its length-only table.

1.6 Palindrome Partitioning II

Problem Statement. Given a string s , partition it into the minimum possible number of substrings such that every substring is a palindrome. Return the **minimum number of cuts** needed (one fewer than the number of pieces).

Pattern Recognition

This problem layers **two** DP tables, each from a different chapter's family: an interval DP table (Section 1.1's style) precomputing *which substrings are palindromes*, feeding into a 1D ending-here table (the DP on LIS chapter's style) computing the minimum cuts.

Step 1: Precomputing palindromic substrings (interval DP)

Let $isPal(i, j)$ be true exactly when $s[i..j]$ is a palindrome. The classic interval recurrence:

$$isPal(i, j) = (s[i] = s[j]) \wedge (j - i < 2 \vee isPal(i+1, j-1)).$$

A single character ($i = j$) or two equal characters ($j = i+1, s[i] = s[j]$) is trivially a palindrome; otherwise, $s[i..j]$ is a palindrome exactly when its outer characters match *and* the inner substring $s[i+1..j-1]$ is itself a palindrome. This table is filled by increasing range length, exactly like every other interval DP in this chapter.

Step 2: Minimum cuts (1D ending-here DP)

Let $cuts[i]$ be the minimum cuts needed for the prefix $s[0..i]$. For every j from 0 to i where $s[j..i]$ is a palindrome (looked up from Step 1's table in $O(1)$), this palindrome can be the **last piece** of the partition, requiring whatever cuts were needed for the prefix before it, plus one more cut to separate this final piece (or zero cuts if $j = 0$, since the whole prefix is one single piece with no preceding pieces):

$$cuts[i] = \min_{j \leq i, isPal(j, i)} (j = 0 ? 0 : cuts[j-1] + 1).$$

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int minCutsPalindromePartition(string s) {
5     int n = s.size();
6     vector<vector<bool>> isPal(n, vector<bool>(n, false));
7
8     for (int len = 1; len <= n; len++) {
9         for (int i = 0; i + len - 1 < n; i++) {
10             int j = i + len - 1;
11             if (s[i] == s[j] && (len <= 2 || isPal[i + 1][j - 1])) {
12                 isPal[i][j] = true;
13             }
14         }
15     }
16

```

```

17     vector<int> cuts(n, INT_MAX);
18     for (int i = 0; i < n; i++) {
19         for (int j = 0; j <= i; j++) {
20             if (isPal[j][i]) {
21                 int prevCuts = (j == 0) ? 0 : cuts[j - 1] + 1;
22                 cuts[i] = min(cuts[i], prevCuts);
23             }
24         }
25     }
26     return cuts[n - 1];
27 }
28
29 int main() {
30     cout << minCutsPalindromePartition("aab") << endl; // 1
31     return 0;
32 }

```

Dry run

Dry Run. Take $s = \text{"aab"}$.

Step 1: *isPal* is true for every single character $((0,0), (1,1), (2,2))$, for $(0,1)$ ("aa"), and false for $(1,2)$ ("ab") and $(0,2)$ ("aab", outer characters a, b mismatch).

Step 2: $cuts[0] = 0$ (single character "a", no cuts needed). $cuts[1]$: $j = 0$ gives "aa", a palindrome, contributing 0; $j = 1$ gives "a" alone, contributing $cuts[0] + 1 = 1$. Minimum: $cuts[1] = 0$. $cuts[2]$: $j = 2$ gives "b" alone, contributing $cuts[1] + 1 = 1$; no other j gives a palindrome ending at index 2. $cuts[2] = 1$. Answer: 1 cut, partitioning "aa" | "b".

Complexity Analysis

Time Complexity. $O(n^2)$ for Step 1 (interval DP, filled in $O(1)$ per cell here since no further split-point search is needed – only the immediately smaller inner range is checked), plus $O(n^2)$ for Step 2 (an $O(n)$ scan per index): $O(n^2)$ overall. **Space Complexity.** $O(n^2)$ for *isPal*, plus $O(n)$ for *cuts*.

Key Takeaway

Not every interval DP needs the full $O(n)$ split-point search this chapter opened with – *isPal* is an interval table whose recurrence only ever looks at *one* smaller inner range, not a search over all of them, because a palindrome's defining property (matching outer characters) only has one way to shrink. Recognizing when an interval recurrence collapses to a single inner lookup (rather than a full split-point scan) is what keeps this problem at $O(n^2)$ instead of the $O(n^3)$ every other section in this chapter needs.

1.7 Partition Array for Maximum Sum

Problem Statement. Given an array *arr* and an integer k , partition *arr* into contiguous subarrays each of length at most k . Within each chosen subarray, replace every element with that subarray's **maximum** value. Maximize the sum of the resulting array.

Pattern Recognition

This closing problem of the chapter simplifies Section 1.1's template in one specific way: because every partition is bounded to length at most k , the search over "where to place the next partition boundary" is bounded to k choices, not a full $O(n)$ scan over every possible

split – collapsing the usual $O(n^3)$ interval-DP cost down to $O(n \cdot k)$.

Deriving the recurrence

Let $f(i)$ be the maximum achievable sum for the suffix $arr[i..n-1]$. The *first* partition starting at index i can have any length L from 1 to k (as long as it fits within the array): choosing length L replaces $arr[i..i+L-1]$ with L copies of its maximum value, contributing $L \times \max(arr[i..i+L-1])$, plus whatever the best suffix sum is afterward:

$$f(i) = \max_{1 \leq L \leq k, i+L \leq n} \left(L \times \max(arr[i..i+L-1]) + f(i+L) \right), \quad f(n) = 0.$$

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int partitionRecursive(int i, vector<int>& arr, int k) {
5     int n = arr.size();
6     if (i == n) return 0;
7
8     int best = INT_MIN, maxVal = INT_MIN;
9     for (int len = 1; len <= k && i + len <= n; len++) {
10         maxVal = max(maxVal, arr[i + len - 1]);
11         int candidate = len * maxVal + partitionRecursive(i + len, arr,
12             k);
13         best = max(best, candidate);
14     }
15     return best;
16 }
17
18 int main() {
19     vector<int> arr = {1, 15, 7, 9, 2, 5, 10};
20     int k = 3;
21     cout << partitionRecursive(0, arr, k) << endl; // 84
22     return 0;
23 }
```

Approach 1 Complexity

Time. $O(k^n)$ in the worst case (k branches per call, n levels of recursion). **Space.** $O(n)$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int partitionMemo(int i, vector<int>& arr, int k, vector<int>& memo) {
5     int n = arr.size();
6     if (i == n) return 0;
7     if (memo[i] != -1) return memo[i];
8
9     int best = INT_MIN, maxVal = INT_MIN;
```

```

10     for (int len = 1; len <= k && i + len <= n; len++) {
11         maxVal = max(maxVal, arr[i + len - 1]);
12         int candidate = len * maxVal + partitionMemo(i + len, arr, k,
13             memo);
14         best = max(best, candidate);
15     }
16     return memo[i] = best;
17 }
18 int main() {
19     vector<int> arr = {1, 15, 7, 9, 2, 5, 10};
20     int n = arr.size(), k = 3;
21     vector<int> memo(n, -1);
22     cout << partitionMemo(0, arr, k, memo) << endl; // 84
23     return 0;
24 }

```

Approach 2 Complexity

Time. $O(n \cdot k)$ – n states, each scanning at most k partition lengths. **Space.** $O(n)$ memo + $O(n)$ recursion stack.

Approach 3: Tabulation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int partitionTabulation(vector<int>& arr, int k) {
5      int n = arr.size();
6      vector<int> dp(n + 1, 0);
7
8      for (int i = n - 1; i >= 0; i--) {
9          int maxVal = INT_MIN;
10         for (int len = 1; len <= k && i + len <= n; len++) {
11             maxVal = max(maxVal, arr[i + len - 1]);
12             dp[i] = max(dp[i], len * maxVal + dp[i + len]);
13         }
14     }
15     return dp[0];
16 }
17
18 int main() {
19     vector<int> arr = {1, 15, 7, 9, 2, 5, 10};
20     cout << partitionTabulation(arr, 3) << endl; // 84
21     return 0;
22 }

```

Dry run

Dry Run. Take $arr = [1, 15, 7, 9, 2, 5, 10]$, $k = 3$.

The optimal partition: $[1, 15, 7]$ (length 3, max 15, contributes $3 \times 15 = 45$), $[9]$ (length 1, contributes 9), $[2, 5, 10]$ (length 3, max 10, contributes $3 \times 10 = 30$). Total: $45 + 9 + 30 = 84$, matching $dp[0] = 84$. Note the middle group is a single element, length 1 – the DP correctly recognizes that extending it to length 2 or 3 (absorbing 2 or 2, 5) would force those smaller values up to 9 as well, costing more than it gains.

Approach 3 Complexity

Time. $O(n \cdot k)$. **Space.** $O(n)$ for the dp array. Since every state depends only on the next k states, a further optimization could keep just a sliding window of k values instead of the full array, but the saving from $O(n)$ to $O(k)$ is rarely worth the added complexity when $k \ll n$ is already small.

Key Takeaway

A length cap on partitions is the cleanest possible way an interval DP can avoid its usual $O(n)$ split-point search: when the next boundary can only be placed within a fixed-size window, the search collapses from a full scan over the entire remaining array to a scan bounded by that window size – turning what would otherwise be $O(n^2)$ or $O(n^3)$ into $O(n \cdot k)$.